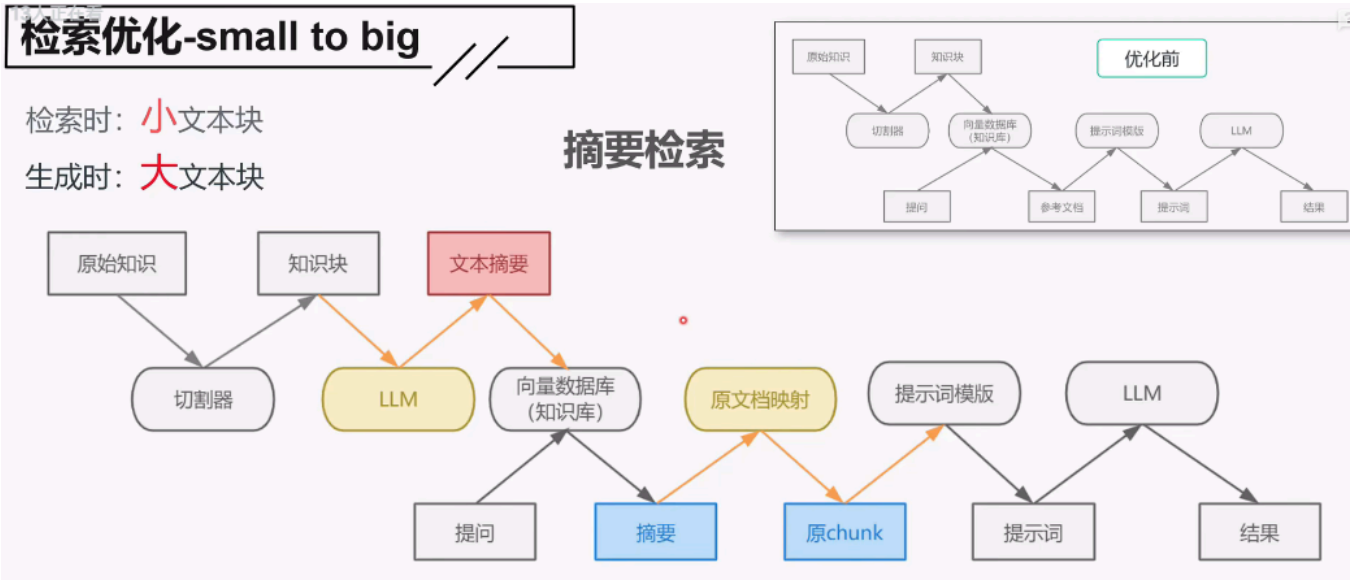
RAG优化技巧

检索优化-Small to big

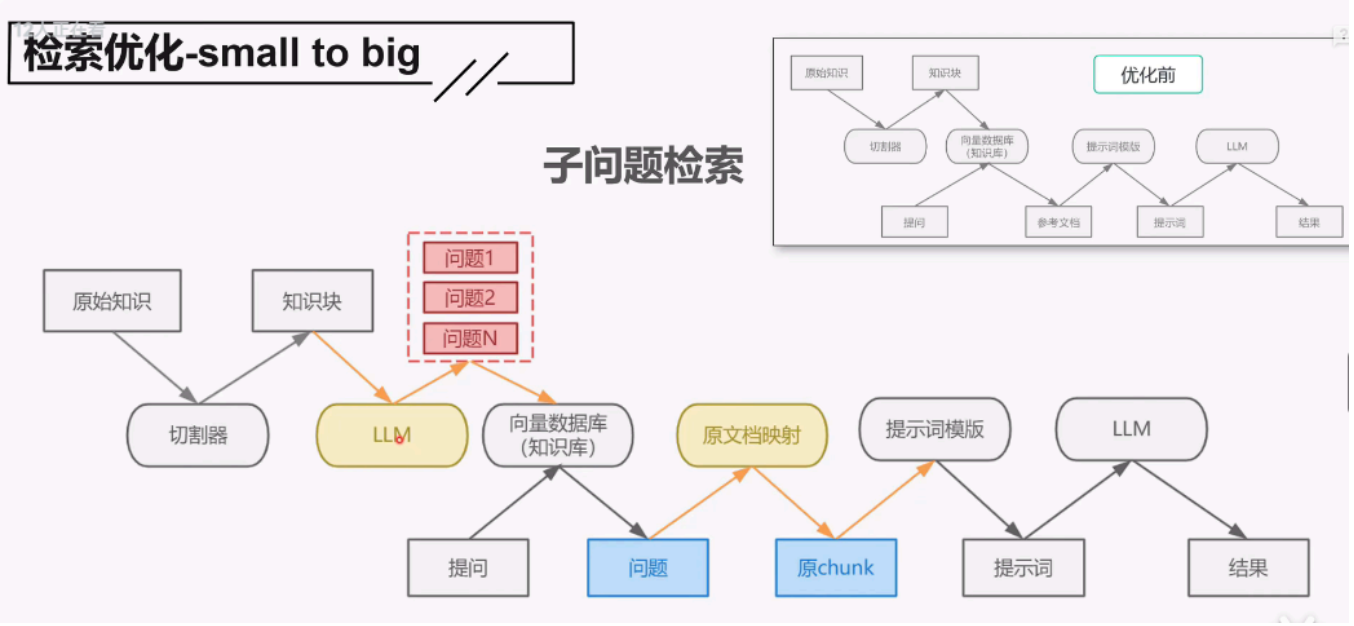
> 💡 检索时：小文本块；生成时：大文本块

摘要检索



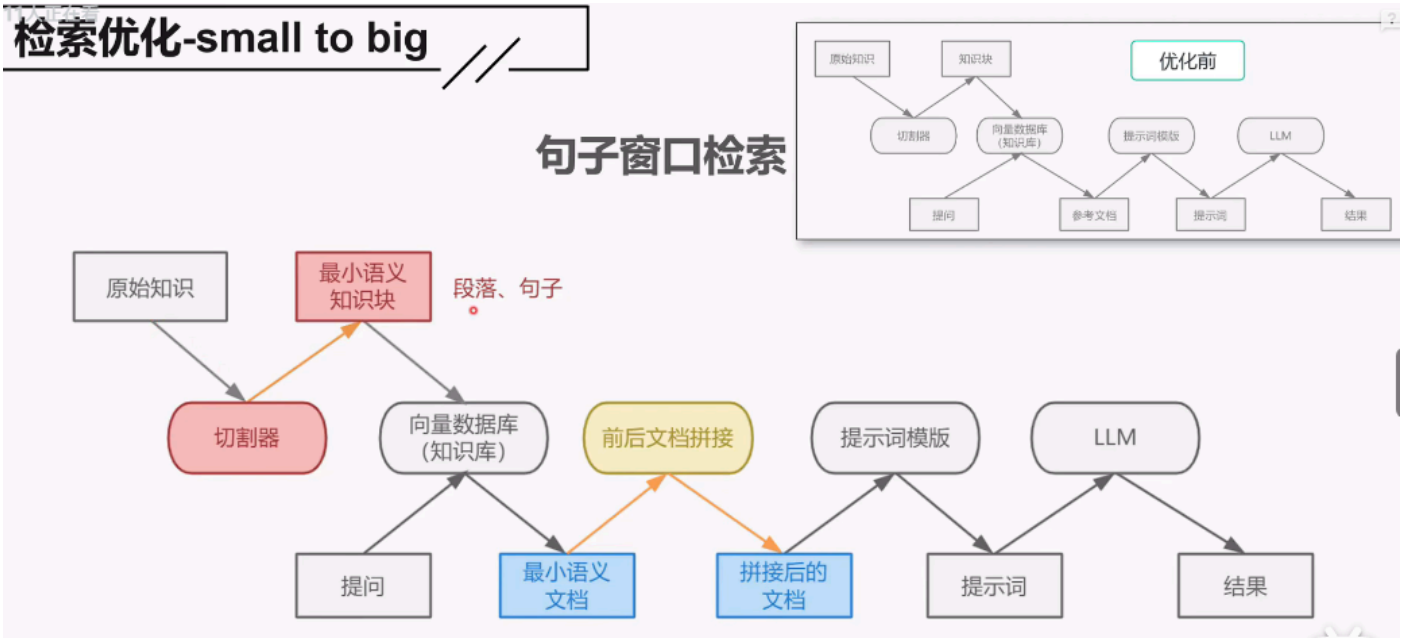
1. 知识块切分大一点
2. 大模型总结摘要存入数据库
3. 根据提对摘要进行匹检索
4. 得到匹配的摘要后，用摘要匹配原来知识块生成答案

子问题检索



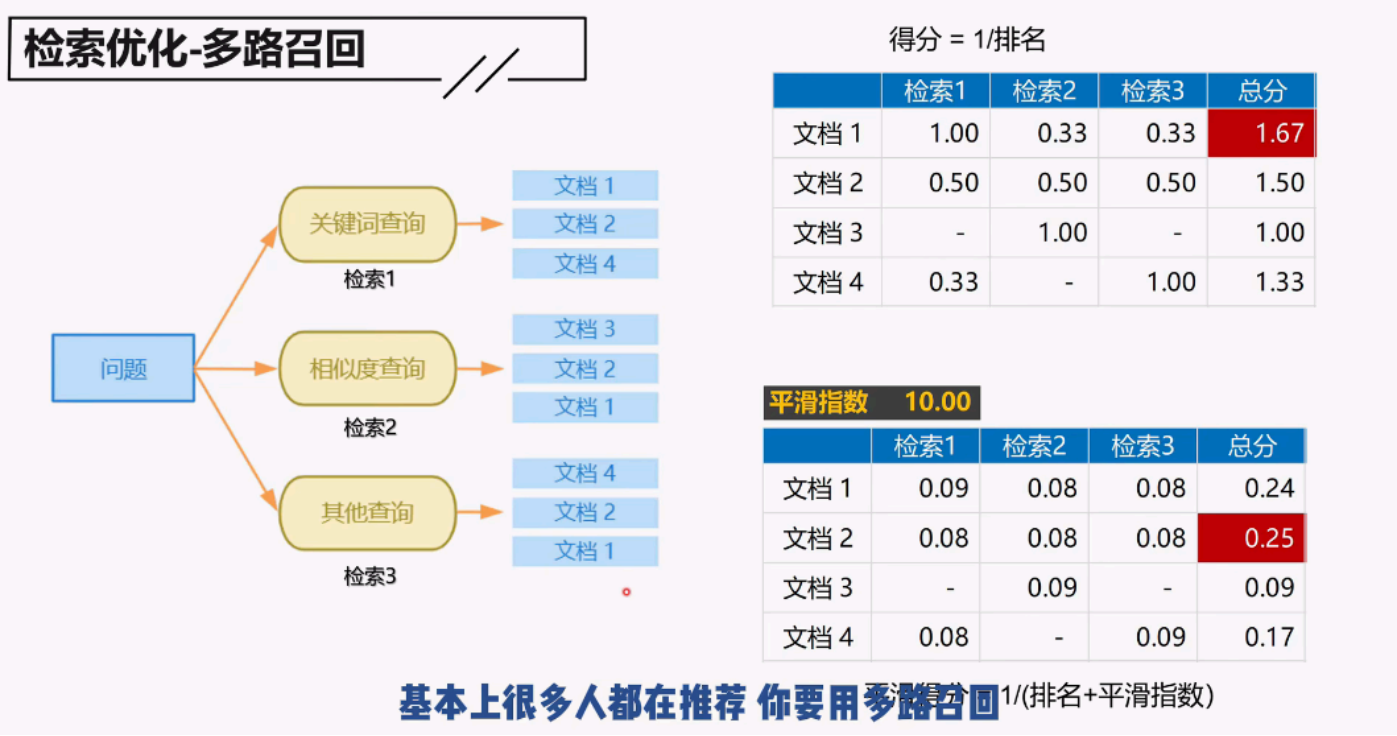
1. 针对知识块，大模型生成子问题存入数据库
2. 用户问题针对子问题进行检索找到匹配问题
3. 针对问题找到文本，拿着匹配的知识块进行答案生成

句子窗口检索



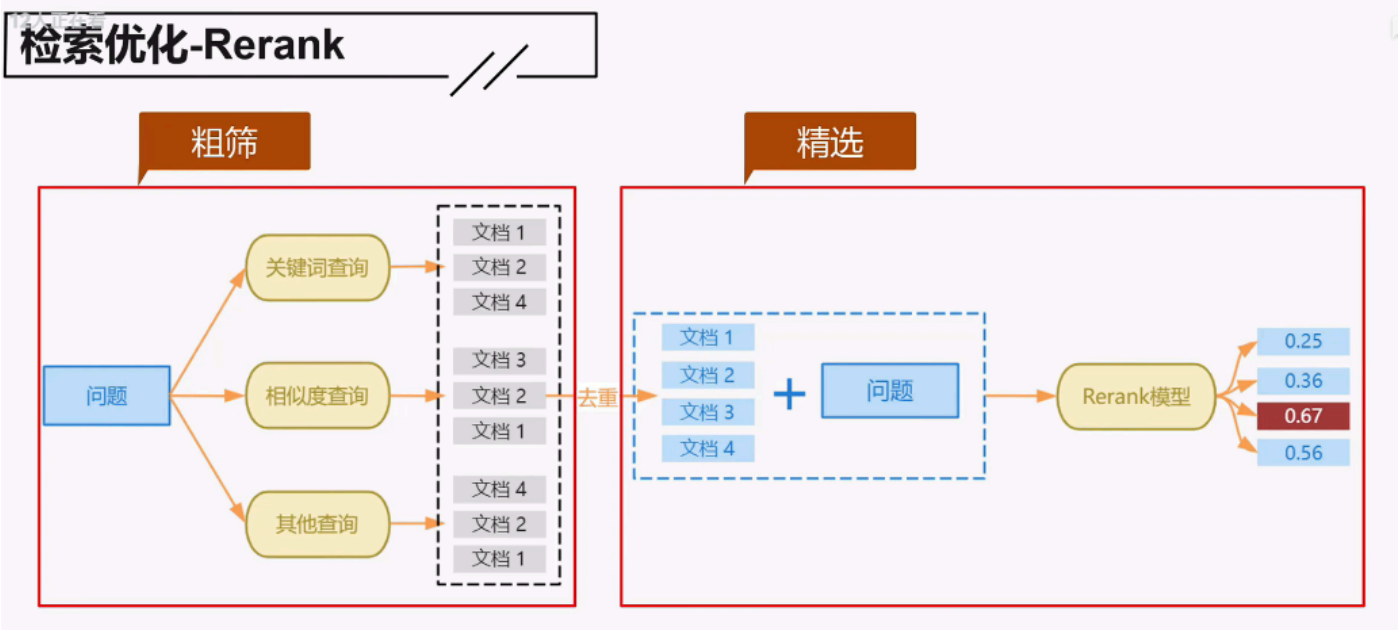
1. 切割文本为最小单位（段落/句子）
2. 应用中拿到匹配的句子后，前后文档进行拼接

多路召回



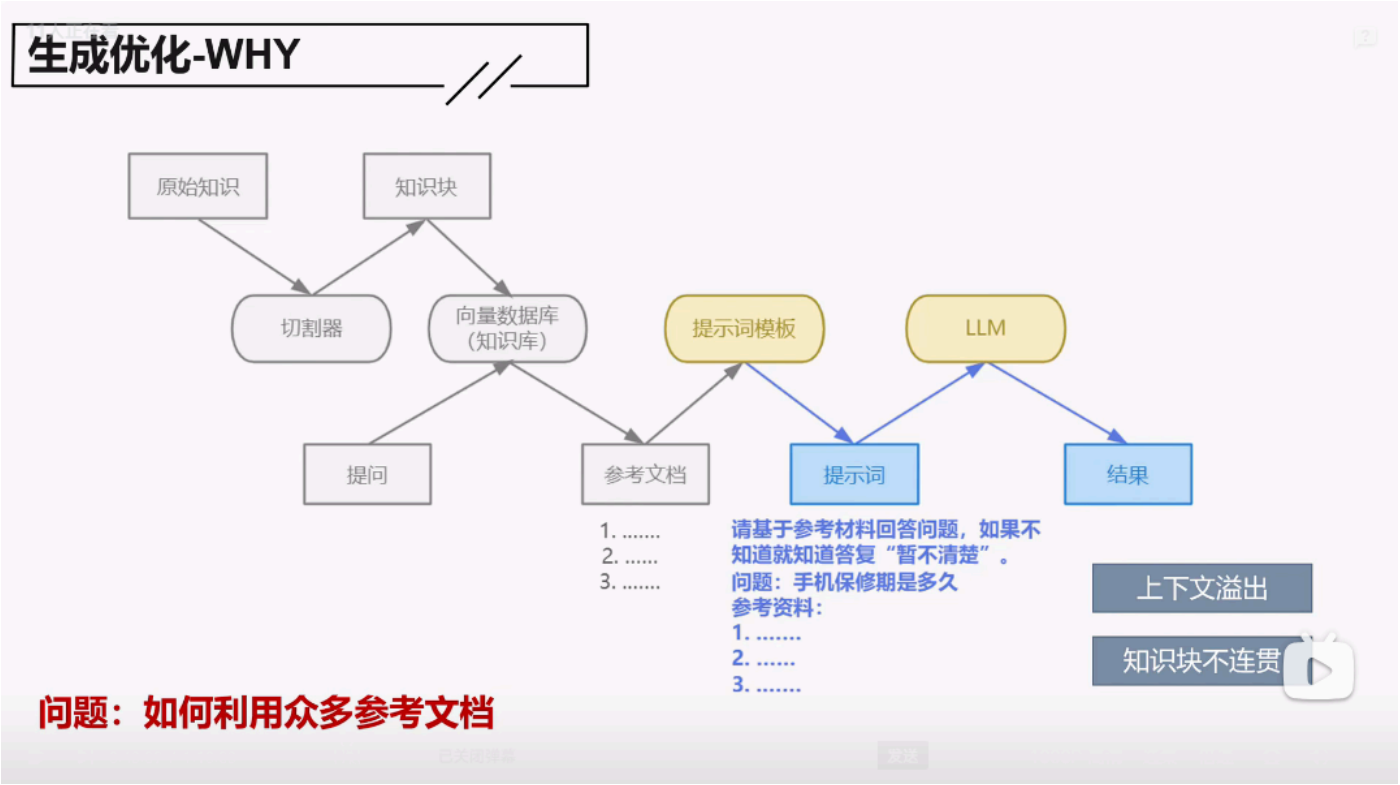
1. 运用多种方法检索，给出答案
2. 根据文档在不同检索的得分进行加总
3. 根据得分进行平滑处理

Rerank：耗费资源

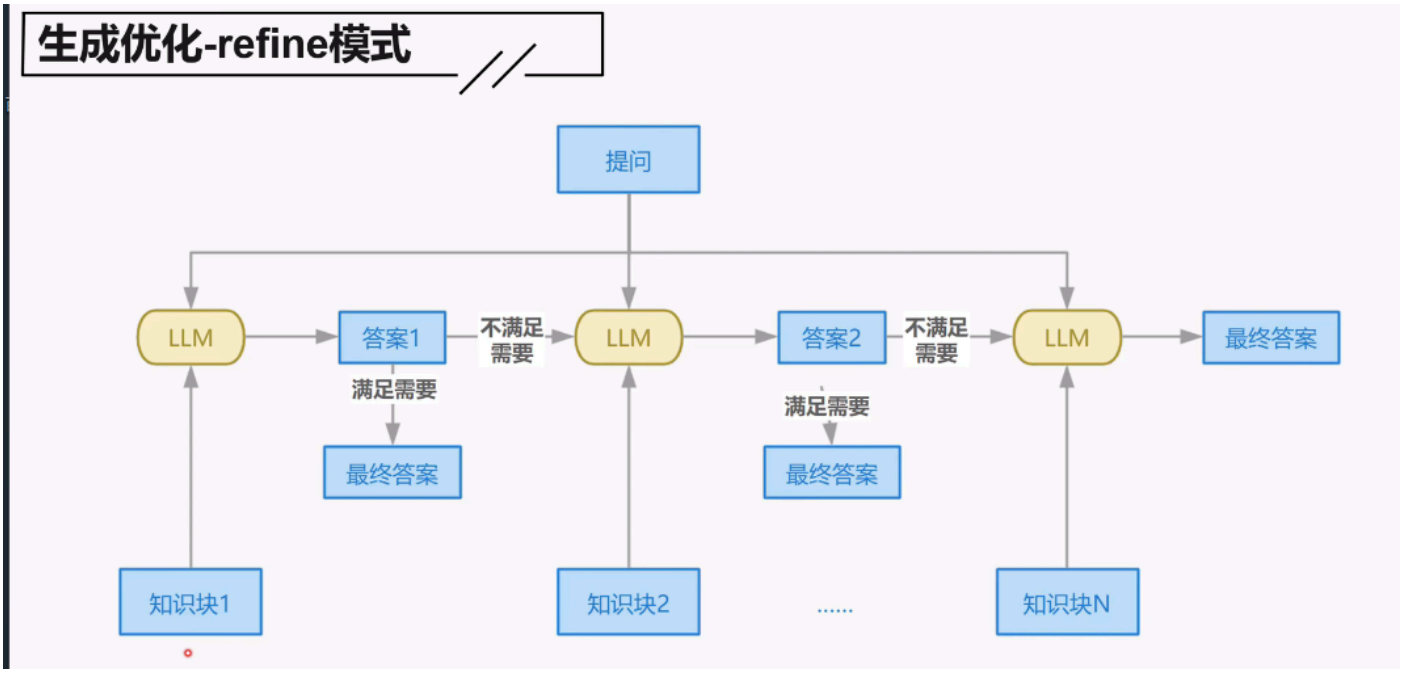


1. 不同查询方法与文档匹配
2. 文档与问题计算相关性得分
3. 选择相关性较高的文档作为上下文

生成优化：如何利用众多参考文档

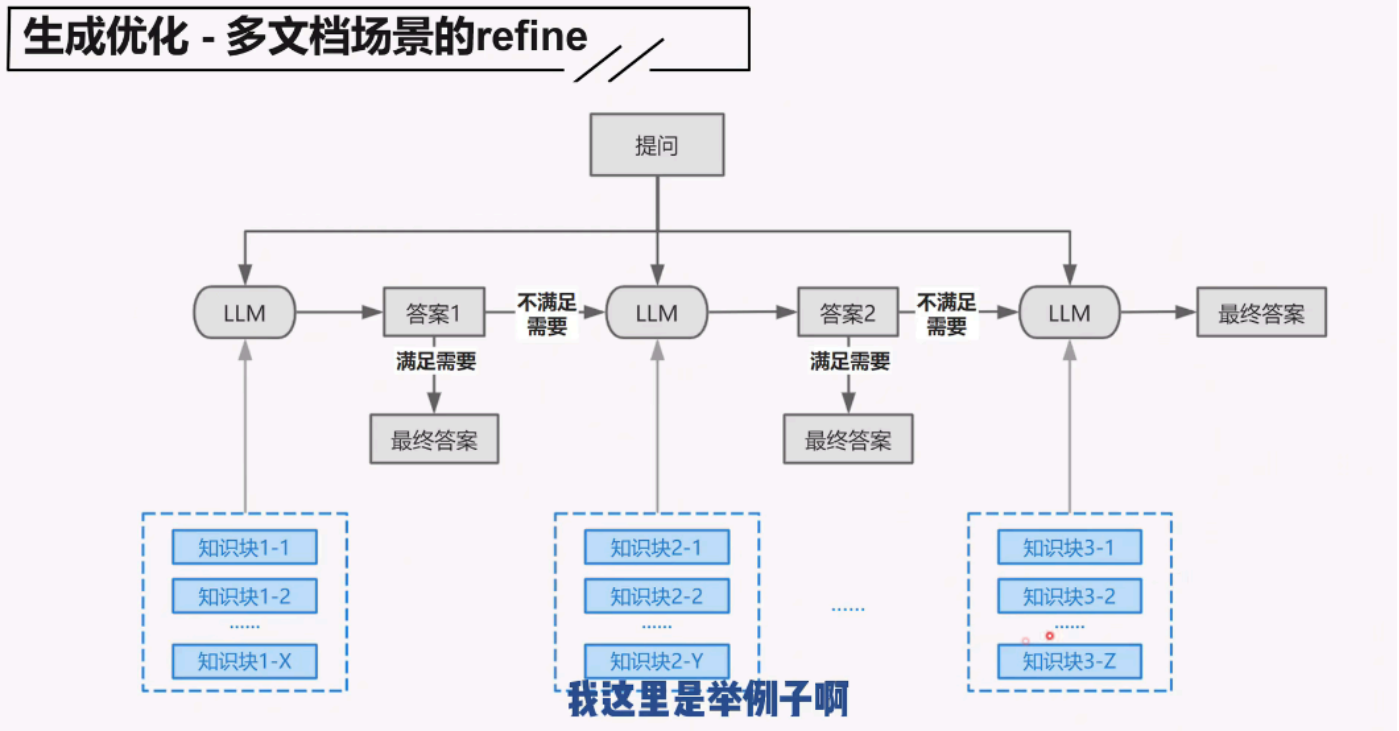


Refine模式



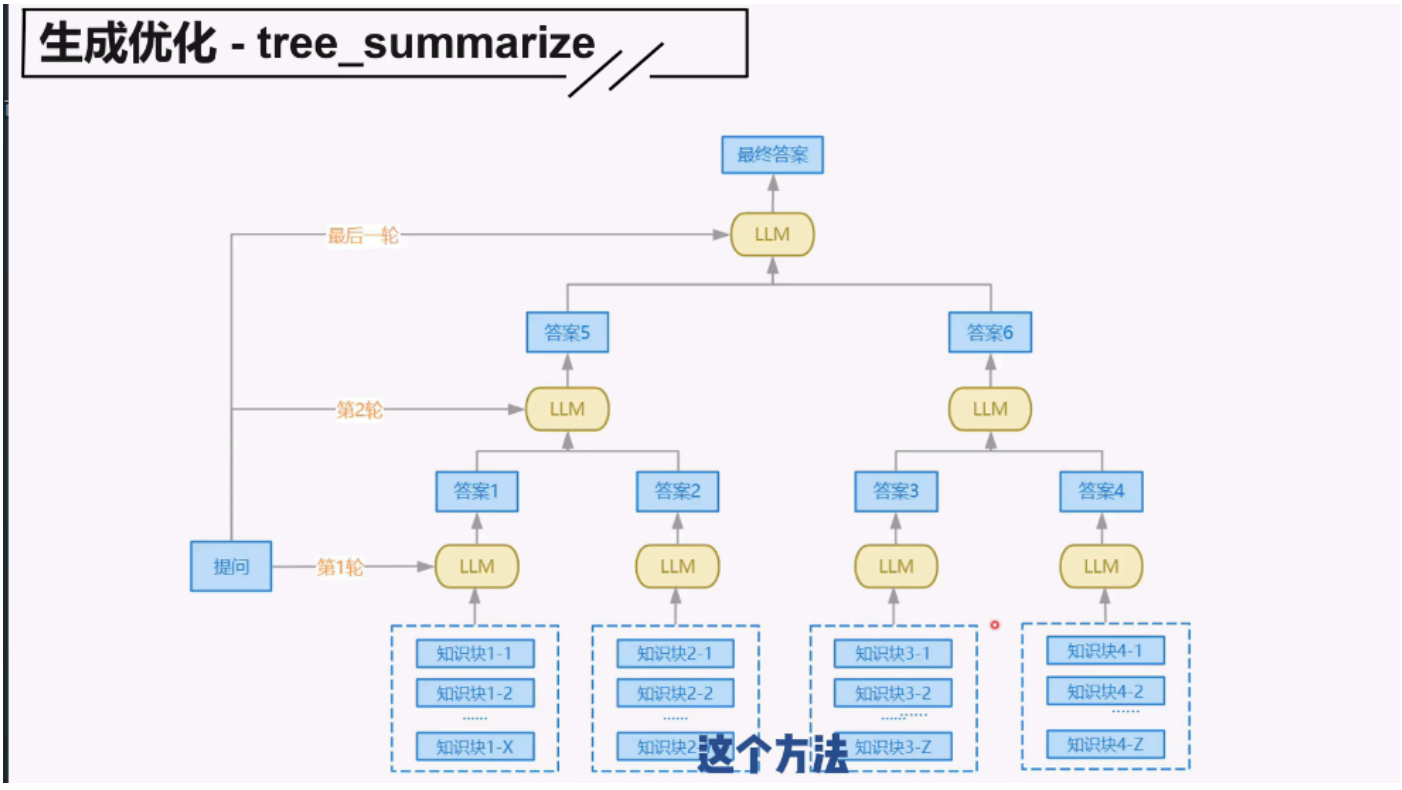
1. 遍历迭代知识块
2. 看结果是否为想要的答案
3. 避免上下文溢出

多文档场景下的Refine



1. 打包知识块迭代
2. 打包方式：聚类，相似性分组

Tree_summarize



1. 检索知识块，分组丢给LLM得到答案

2. 对答案进行打包后丢给LLM
3. 迭代答案只能生成一个包最后丢给LLM
4. 将答案当成参考文档

改写提问

1. 针对用户提问无法进行检索
2. 提问不规范：语义模糊、语句不通顺
3. 多轮对话：历史回答对提问改写
4. 复杂提问：拆解成多个问题，分别提问RAG

特征/需求	基于规则的改写	基于模型的改写
问题复杂度	简单、规则明确的情况	复杂、多样化的问题
可控性	高，可精确控制	低，可自由生成，但可能不完全可控
计算资源	低，快速	高，需要更多计算资源
灵活性	低，规则固定	高，可以处理各种变体
应用场景	实时性高、规则化强的场景（如FAQ、客服机器人）	自然对话系统、多领域应用、复杂问题求解

合理利用元数据

什么是元数据：对文档的描述，如类型、作者、创建日期、来源等

检索前过滤：如：剔除无关文档、检索最近文档

检索中评估相关性：如：重要文档给予更高的权重加成

检索后重排：如：重要文档排前面

生成时提升用户体验：提供参考文档来源，增加可信度